Delayed Gradients in CPU-Based Neural Network Training: An Operating Systems Study of Bounded Staleness

Kaustubh Upadhya, Dhruv Pekhale, Jeet Mahajan, Tejas Tandon
Indian Institute of Information Technology, Sri City
Operating Systems Course Project

***Abstract*—Splitting a neural network across CPU workers does not remove its forward and backward dependencies. We study a two-worker C++ runtime that permits several examples in flight and discards hidden-layer gradients whose version age exceeds a configured limit. Each worker owns one layer, and bounded queues carry activations, derivatives, and completion results. The workload is a 784-32-10 multilayer perceptron trained on 5,000 MNIST examples per epoch and evaluated on 1,000 test examples. Nine recorded five-epoch runs show delay four averaging 20.52 seconds, compared with 28.28 seconds sequentially. A new sweep of the project implementation, comprising 24 optimized runs on a documented CPU host, gives 2.408 seconds for delay four and 3.494 seconds sequentially. The delayed setting accepts 75.03% of hidden updates, so its elapsed-time gain combines scheduling changes with omitted computation. Source inspection also shows that the hidden worker recomputes the ReLU mask at gradient arrival; accepted updates therefore combine an older downstream derivative with the current local Jacobian. We explain the ownership and queue protocol, distinguish version age from elapsed delay, and compare throughput with time to an accuracy target. The results support a workload-specific speed-quality tradeoff, with no claim of an optimal universal delay or equal-work parallel speedup.**

***Index Terms*—delayed gradients, bounded staleness, operating systems, producer-consumer queues, model parallelism, CPU training**

I. Introduction

The question behind this project is: how much synchronization waiting can CPU workers remove while keeping neural-network training useful? A straightforward division of a network into layers gives different threads different parameter arrays. It does not remove the data dependencies between those arrays. A hidden layer produces activations needed by the output layer; the output layer produces the error signal needed to update the hidden layer. If only one example may occupy the system, the two threads repeatedly hand control to one another. The program is concurrent, but its useful arithmetic remains mostly serial.

Allowing several examples in flight changes the schedule. The hidden worker can compute another forward pass before the previous error signal returns. By then, earlier gradients may have changed its parameters. The arriving signal is still produced by backpropagation, but it refers to an older hidden-layer version. This is the delayed-gradient setting. Its purpose is to relax temporal coordination while retaining an explicit account of the age of information.

The project uses a small CPU multilayer perceptron (MLP) so that the runtime can be understood without an accelerator framework. The hidden layer has 784 inputs and 32 ReLU units; the output layer has ten logits. Worker 0 owns the hidden parameters and Worker 1 owns the output parameters. A coordinator bounds the number of submitted examples that have not completed. Messages carry an example identifier and a hidden-layer origin version. Worker 0 accepts or discards each returning gradient according to a user-selected maximum age. The policy does not undo the output update when it discards a hidden gradient.

This last detail matters for performance analysis. Discarding a gradient avoids the hidden weight update, which touches a much larger matrix than the output layer does. A shorter run can therefore reflect less arithmetic as well as more overlap. It is misleading to describe every elapsed-time reduction as synchronization removed by parallelism. The completed-example count also remains high when the hidden layer receives few updates. Training quality and useful update counts must accompany throughput.

The evidence covers three stages of the project: a one-epoch pilot, a recorded five-epoch comparison, and a new sweep of the submitted implementation with documented build and machine settings. The pilot contains repeated baseline and delay experiments. The main comparison supplies learning curves and target times. The new sweep tests the executable now included in the submission and records complete logs, source hashes, and dataset hashes. Each set is analyzed separately because its duration, instrumentation, and available build information differ.

The project contributes a concrete owner-thread runtime, an explicit staleness policy, and measurements that connect synchronization choices to learning quality. It adapts established delayed-gradient ideas rather than proposing a new optimization algorithm. Its operating-systems content lies in message lifetime, mutex and condition-variable coordination, finite buffers, work admission, stage imbalance, shutdown, and measurement scope.

We first place the design in the literature, then describe its mathematics and execution protocol. The results separate completed examples from accepted hidden updates and distinguish a faster training window from faster attainment of useful quality. The submission repository contains the source, raw mea-

surements, figures, report source, and reproduction commands: https://github.com/kaustubh-dot/delayed-gradient-cpu-mlp.

II. Literature and design context

A. Asynchronous optimization and bounded consistency

HOGWILD! allows processors to update shared parameters without locking and analyzes conditions under which sparse update conflicts remain manageable [1]. It establishes a useful historical motivation for avoiding coordination on every update. Our design has different memory semantics: dense parameter arrays have one writer each, and communication uses mutex-protected queues. We neither implement lock-free shared writes nor inherit sparse-conflict guarantees for this dense MLP.

Stale synchronous parallel (SSP) computation permits bounded differences between worker progress clocks and forces faster workers to wait when the limit is exceeded [2]. This separates strict synchronization from unconstrained asynchrony. Our age limit is related in spirit but operates at a different point. It tests a returned hidden gradient against the current hidden version and may discard it. It does not enforce the SSP parameter-server consistency model or bound a gap between independent worker clocks. A worker-progress clock, a parameter version, and a wall-clock delay are distinct quantities.

B. Decoupling backpropagation with real delayed signals

Huo et al. introduce decoupled parallel backpropagation using delayed gradients and establish convergence to critical points for their stochastic methods under the paper's assumptions [3]. Their experiments use deeper convolutional networks and a different execution setting. We borrow the idea that backward information can be temporally delayed. Our two-layer CPU implementation, owner-thread rule, and age-based rejection policy are engineering choices for this course project. The published theorem does not by itself prove convergence of our runtime.

Features Replay addresses the memory and accuracy costs of decoupling through a parallel-objective formulation and replay of intermediate features [4]. It shows that retaining information from earlier execution is part of the algorithm, not merely a buffer-management detail. Our hidden worker retains a copy of each in-flight input, while the activation travels to the output worker. It does not implement the Features Replay objective or repeatedly train from a replay buffer. The input cache supports later local backward computation; it does not preserve the origin ReLU mask.

Fully Decoupled Neural Network Learning Using Delayed Gradients (FDG) seeks to break forward, backward, and update locking, and introduces gradient shrinking to reduce stale-gradient effects [5]. Its published journal version has four authors and appears in the 2022 volume of IEEE Transactions on Neural Networks and Learning Systems. Our pipeline still requires an upstream activation before output computation. It should not be called a fully decoupled implementation of FDG. We use a binary age gate rather than the paper's gradient-shrinking procedure.

Diversely Stale Parameters (DSP) introduces layer-wise staleness and studies model-parallel training with unequal device speeds [6]. That work is especially relevant to the distinction between delay at different modules and a single global training timestamp. Our hidden age counter measures only accepted hidden updates since an activation was produced. It does not measure the age of every parameter used to produce the downstream derivative, and it is not a full implementation of DSP.

Peng et al. combine recomputation with weight prediction in DTRP to address retained-state memory costs and the additional delay caused by recomputation [7]. This places two possible strategies in context: store the forward information until the gradient arrives, or recompute it later while accounting for changed weights. Our small network stores inputs and recomputes the hidden preactivation during local backward computation. It performs no weight prediction, batch compensation, or correction of the returned downstream signal. This simple choice makes the runtime easier to inspect, but it would need reconsideration for deep networks with large activation tensors.

C. Recent work and the limits of analogy

Ringmaster ASGD selectively ignores sufficiently stale gradients and proves time-complexity results under heterogeneous, time-varying worker computation [8]. Its mathematical setting uses workers computing stochastic gradients of a common optimization objective. Our workers own different layers and exchange partial derivatives. The shared use of rejection does not make the methods equivalent. In particular, our age threshold is chosen as a small experimental control, not from Ringmaster's theoretical rule.

The 2026 dissertation [9] develops this asynchronous optimization line across homogeneous and heterogeneous data. It provides recent context through the literature search date of October 8, 2026, but we identify it as a dissertation rather than a new peer-reviewed conference result. The broader lesson for this project is to evaluate progress in elapsed time while stating the computation and delay assumptions. None of these references establishes that a delay of four is optimal for a two-stage CPU MLP.

D. Alternatives considered in the initial survey

Pipeline parallelism schedules micro-batches across stages to hide dependencies; GPipe provides an established example [10]. Dependencies remain, and stage balance, bubbles, and activation storage affect efficiency. Synthetic gradients instead estimate a missing downstream gradient using a learned model, as in decoupled neural interfaces [11]. Local learning substitutes a local objective. Both alter the learning signal and introduce questions beyond its age. Local SGD gives each worker a complete model and periodically averages replicas [12]; its principal tradeoff is communication frequency versus replica drift.

TABLE I
DESIGN-SPACE CONTEXT; ALTERNATIVES ARE NOT IMPLEMENTED HERE.

Approach	Coordination decision
Delayed gradients	Use an older real derivative; track or limit its age.
Pipeline	Schedule another ready micro-batch while dependencies remain.
Synthetic/local	Predict the downstream signal or replace it with a local loss.
Local SGD	Train full private replicas between averaging points.

These alternatives target different forms of coordination. The selected delayed-gradient approach keeps genuine backpropagation and one partitioned model, so the effect of stale information can be studied directly through version counters. Table I summarizes the distinction. We do not benchmark the alternatives; the supplied artifacts and new experiments concern delayed gradients only.

III. MATHEMATICAL MODEL AND GRADIENT SEMANTICS

A. The learning task

MNIST is a handwritten-digit benchmark with 28 by 28 grayscale images [13]. We normalize byte intensities to $[0, 1]$ and flatten each image to $x \in \mathbb{R}^{784}$. The model computes

$$a_0 = W_0 x + b_0, \qquad h = \mathrm{ReLU}(a_0), \tag{1}$$

$$z = W_1 h + b_1, \qquad p_k = \frac{\exp(z_k)}{\sum_j \exp(z_j)}, \tag{2}$$

$$\ell(x, y) = -\log p_y. \tag{3}$$

Here $W_0 \in \mathbb{R}^{32\times 784}$ and $W_1 \in \mathbb{R}^{10\times 32}$. There are 25,450 trainable scalars, including 42 biases. The model is deliberately small. Its purpose is to expose a scheduling tradeoff rather than compete with MNIST classification systems.

For one example, the output derivative is $\delta_1 = p - e_y$. Before updating W_1, the output owner computes

$$d_h = W_1^\top \delta_1, \tag{4}$$

$$\delta_0 = d_h \odot \mathbf{1}[a_0 > 0], \tag{5}$$

$$\nabla W_1 = \delta_1 h^\top, \qquad \nabla W_0 = \delta_0 x^\top. \tag{6}$$

Bias derivatives are the corresponding local deltas. The implementation computes the softmax loss by subtracting the maximum logit and using the log-sum-exp expression. This avoids overflow without changing the derivative. It computes d_h from the pre-update output weights; otherwise the returned signal would not match the output computation used for that example.

B. Versions and the acceptance rule

Let v_0 be the hidden layer's accepted-update count. When Worker 0 produces h for an example, it attaches origin version $o = v_0$. When the derivative returns, its age is

$$a = v_0 - o. \tag{7}$$

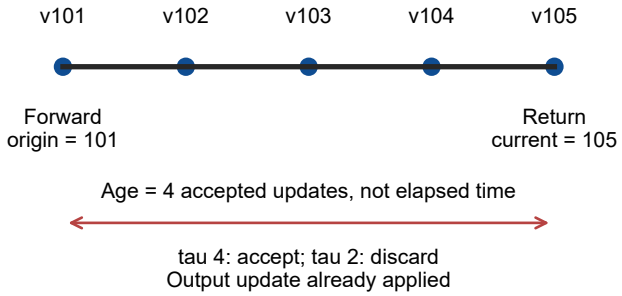


Fig. 1. A conceptual version trace. The four intervening events are accepted hidden updates. This illustration is not a measured scheduling timeline.

An accepted gradient increments v_0 by one; a rejected gradient leaves it unchanged. Thus age counts parameter changes, not examples submitted, output updates, queue operations, or elapsed milliseconds. The policy is

$$\theta_0 \leftarrow \theta_0 - \eta g_0 \quad \text{if } a \le \tau_{\max}. \tag{8}$$

Every processed activation updates the output layer whether or not its hidden gradient later passes this test. The decision is local to Worker 0, which can compare and update the version without an atomic parameter operation.

The familiar delayed-SGD equation $\theta_{t+1} = \theta_t - \eta g(\theta_{t-a})$ is useful intuition, but it does not exactly describe this modular program. Hidden activations come from θ_0^o, while the output computation uses θ_1^s when Worker 1 processes them. The returned vector d_h is the derivative of the downstream loss with respect to that activation at the output state s. Those layer versions need not have existed together as a complete model snapshot.

C. The local Jacobian used at arrival

The hidden worker caches the input vector x, not the forward activation or ReLU mask. If it accepts a returned derivative, `DenseLayer::backward` recomputes $W_0 x + b_0$ using the hidden parameters currently held by the worker. Its local delta is therefore

$$\widetilde{\delta}_0 = d_h^{o,s} \odot \mathbf{1}[W_0^{v_0} x + b_0^{v_0} > 0]. \tag{9}$$

The hidden update uses $\widetilde{\delta}_0 x^\top$ and $\widetilde{\delta}_0$ for the weight and bias derivatives. The downstream signal refers to an earlier activation, while the local ReLU mask refers to the arrival state.

At age zero the hidden parameters have not changed since the forward pass, so the local mask agrees with the forward mask. At positive age they may differ. An accepted update is consequently not guaranteed to equal the full derivative of the original forward computation. It is also not generally a current-model gradient, since the output calculation used the older hidden activation. The source uses genuine backpropagated

TABLE II
MESSAGES USED BY THE RUNTIME. NUMERIC VECTORS CONTAIN FLOATS.

Message	Payload and purpose
Training	ID, label, 784 input values; request hidden forward work.
Activation	ID, label, hidden origin version, 32 values; request output work.
Gradient	ID, origin and producer versions, loss, 32 derivatives; return the downstream signal.
Result	ID, age, acceptance flag, loss, layer versions; release admission credit.

downstream information, but combines it with a recomputed local Jacobian. This distinction is central to the algorithm being evaluated.

Saving the origin mask would be a reasonable alternative, but it would change the implementation and its learning trajectory. The experiments here retain the project's recomputation rule. No claim of exact stale full-model gradients or convergence follows from the age gate alone.

IV. RUNTIME ARCHITECTURE AND OPERATING-SYSTEMS DESIGN

A. Ownership and message flow

Figure 2 shows two persistent C++17 worker threads and a coordinator on the calling thread. Worker 0 owns the hidden parameters, version counter, and pending-input map. Worker 1 owns the output parameters and its version counter. Each parameter array has one writer. The coordinator performs evaluation only after all admitted examples have completed.

There are three queues. The shared hidden work queue accepts training messages from the coordinator and gradient messages from Worker 1. The activation queue carries hidden outputs to Worker 1. The result queue carries completed hidden decisions to the coordinator. A C++ variant distinguishes training and gradient work in the shared queue. This permits one blocking input stream for the hidden owner without polling two separate queues.

The source calls an example a "batch," but each message contains one image and label. It performs no mini-batch gradient accumulation. Batch IDs remain unique across epochs. Worker 0 copies the input into a pending map, computes an activation at the current hidden version, and sends that activation with its origin version. Worker 1 computes logits, loss, output gradients, and the derivative with respect to the activation. It forms the derivative before changing its weights, then applies the output update and returns the derivative.

Worker 0 finds the matching cached input, verifies that the origin is not a future version, and calculates age. Acceptance triggers local backward recomputation and an update. Rejection skips both operations. Either decision erases the cached input and produces a completion result. The output update is retained when a hidden gradient is discarded.

B. Bounded admission and finite buffers

The coordinator maintains submitted and completed counters. It admits another example only while

$$n_{\text{submitted}} - n_{\text{completed}} < I. \tag{10}$$

A result releases one credit after the hidden decision, including rejection. Thus at most I examples can be outstanding and at most I input records can remain pending. Admission limits work across the complete path rather than only on one edge.

Each queue has capacity I and uses a FIFO `std::queue`, one mutex, and two condition variables. A producer waits while full; a consumer waits while empty. Both waits include closure in their predicates. Predicate waits handle spurious wakeups, and the mutex protects the container and closed flag. The queue releases the lock before notifying the opposite side. Layer arithmetic occurs outside queue locks.

A cycle of bounded queues can deadlock if every stage blocks while trying to write to its successor. The admission rule prevents that cycle in this protocol. Each outstanding example has one queued or currently processed stage message. A worker about to push its message is already processing that example, leaving at most $I-1$ other outstanding examples that could occupy the destination. Capacity I therefore suffices for a valid normal epoch. The coordinator also admits an example only when fewer than I are outstanding. This reasoning depends on the present routing and single message per example; it would need revision for retries, extra messages, or different admission rules.

Bounded queues still matter even when a valid epoch cannot exhaust a destination at a push. They limit storage, coordinate empty-queue waiting, and provide a defined closure mechanism. Full-queue blocking is covered by the queue test, but the training runs contain no occupancy trace or direct measurement of time spent waiting for free capacity. The report consequently does not claim measured backpressure during training.

C. Safety, lifetime, and epoch drains

A hidden forward and hidden update execute on the same owner thread, so their version counter describes the actual hidden parameter state. Output updates also execute serially. Messages contain value-owned vectors instead of references to mutable weights. The pending input survives until the matching derivative is handled, and a duplicate training ID or a missing cache entry causes an error.

Evaluation starts after the coordinator has consumed every result for the epoch. The output update precedes the gradient push, and the hidden decision precedes the result push. Queue mutex release/acquire operations carry this ordering to the coordinator. Once all results have been collected, neither worker has an outstanding example that could change parameters. Workers can remain blocked in empty input queues during read-only evaluation. New training work is submitted only after that evaluation.

The coordinator checks that each result belongs to the current epoch and that its ID has not already completed.

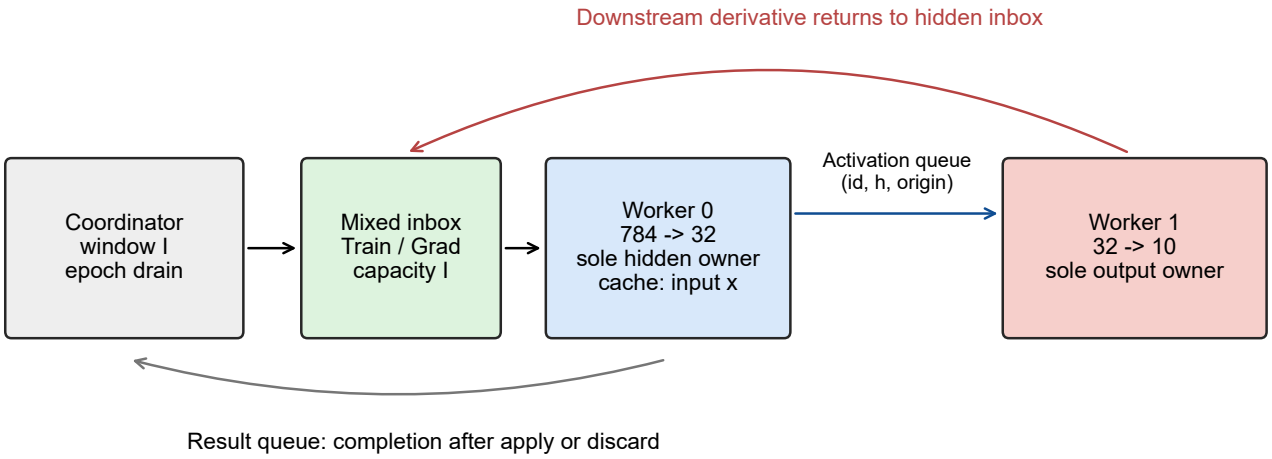


Fig. 2. The submitted runtime. Admission and each queue capacity are bounded by I. The mixed hidden inbox serializes new forward work and returning derivatives on the hidden owner.

These checks protect the completion count from duplicate or misrouted messages. No pending input is intentionally carried between epochs. The training timer starts after shuffling and stops after the final result. It includes input conversion, layer work, admission, queue transfer, and coordination inside the training window.

D. Shutdown and progress

Normal shutdown follows the last epoch drain. The coordinator closes the hidden work queue and joins Worker 0. That worker closes the activation and result queues when its input stream ends, allowing Worker 1 to leave its pop loop and be joined. A closed queue rejects further pushes, but permits already queued values to be drained before a pop reports end of stream.

Worker loops catch exceptions, store them, and close the channels needed to wake other participants. A lifetime guard in the coordinator closes all three queues and joins any joinable workers if an exception interrupts training or thread creation. This prevents an exceptional exit from destroying a joinable thread. The integration suite exercises an invalid-label failure inside the output worker as well as normal shutdown.

Progress still requires the operating system to schedule every participant and the arithmetic to terminate. The design does not guarantee progress under permanent starvation or process failure. Under valid inputs and eventual scheduling, each admitted example produces one output update, one hidden decision, and one result. Count conservation is a useful check of that protocol. The implementation uses neither busy waiting nor detached threads.

V. Algorithm and Implementation Checks

Algorithm 1 gives the coordinator and hidden-owner policies. The sequential executable uses the same dense-layer routines and computes both layer derivatives before updating either layer. The worker executable obtains synchronous behavior with $I = 1$ and $\tau_{\max} = 0$.

The two configurations $(\tau_{\max}, I) = (0, 1)$ and $(0, 8)$ have different semantics. With one outstanding example, no hidden update can intervene between its forward pass and gradient

Algorithm 1 Bounded delayed-gradient training

Initialize both layers and owner threads; $v_0 \leftarrow 0$
for each epoch **do**
  Shuffle selected training indices; $s \leftarrow 0$, $c \leftarrow 0$
  while $c < N$ **do**
    while $s < N$ and $s - c < I$ **do**
      Send input and label with fresh ID; $s \leftarrow s + 1$
    end while
    Receive result; record loss, age, decision; $c \leftarrow c+1$
  end while
  Stop epoch timer; evaluate drained model
end for
Close hidden inbox; join workers
Worker 0, for each mixed-inbox message:
if message is training work **then**
  Cache input x; compute h at v_0
  Send $(\mathrm{ID}, h, y, o = v_0)$ to Worker 1
else
  Retrieve cached input; $a \leftarrow v_0 - o$
  if $a \leq \tau_{\max}$ **then**
    Recompute local ReLU mask at current parameters
    Form local gradient; update hidden layer and v_0
  end if
  Erase input; send result for this ID
end if

return. With eight outstanding examples, several forwards can share one origin. The first accepted gradient advances the hidden version, causing subsequent gradients from that origin to fail the zero-age gate. Their output updates remain applied. High completion throughput can therefore coexist with very few hidden updates.

The C++ unit executable compares dense-layer input derivatives with centered finite differences for linear and ReLU layers on a small synthetic input. It checks that a small SGD update reduces a tested linear objective, that an update increments the version, and that invalid dimensions are rejected. These are focused local checks; they are not an exhaustive finite-difference test of every network weight or a proof of the delayed algorithm.

Queue tests exercise a producer attempting to push into a full queue and a consumer attempting to pop from an empty queue. Closure must release both calls, retain an item already queued, and reject later delivery. A timeout bounds each test. Passing these checks demonstrates the tested cases, while the ownership and admission arguments explain how the training graph uses the queue contract.

Four integration tests generate IDX fixtures containing 5,000 training and 1,000 test records. Sequential and synchronous executions must match training loss, test loss, and accuracy at both checkpoints to the program's six-decimal output precision. Delayed executions check hidden count conservation, one output update per example, maximum returned age at most seven, and full acceptance for limits eight and sixteen. Invalid settings must fail. The worker-error test confirms that a bad label reports an error and exits after joining the workers.

The unit executable and all four integration tests pass. The experiment harness also checks the complete epoch sequence, final count conservation, and observed age bound for every run. These checks support the implementation contracts without claiming to cover every possible interleaving. No race detector, hardware profiler, or model checker is part of this validation.

VI. Theoretical performance and staleness analysis

A. A service-cost model

Let F_0 denote hidden forward cost, B_0 hidden gradient-application cost, and U_1 the output forward/backward/update cost. For N examples, a simple sequential arithmetic model is

$$T_{\text{seq}} \approx N(F_0 + B_0 + U_1). \tag{11}$$

Let r be the hidden acceptance fraction. A delayed run performs approximately N hidden forwards and output updates but only rN hidden gradient applications. Its worker service demands are

$$D_0 = N(F_0 + rB_0), \qquad D_1 = NU_1. \tag{12}$$

A lower-bound intuition is $T_{\text{delay}} \gtrsim \max(D_0, D_1)$ before admission, queue, scheduling, and memory costs. Actual time also includes those overheads and startup/drain bubbles within the epoch. Equations 11 and 12 are a model, not fitted measurements; the supplied logs do not isolate the component costs.

The hidden matrix has 25,088 weights, while the output matrix has 320, a ratio of 78.4. Both the hidden forward and accepted hidden update traverse the larger matrix. This gives a structural reason to expect stage imbalance, but it is not an assertion that Worker 0 takes exactly 78.4 times longer. ReLU masks, memory access, loop optimization, message copies, and fixed overhead all affect elapsed service time.

If U_1 is small, overlapping it with hidden work can remove only a small share of total arithmetic time. By contrast, lowering r can remove a substantial amount of hidden update work. That observation explains why a rejection-heavy mode can be fastest while learning least well. A fair systems interpretation therefore asks whether speed comes from overlapping comparable work, avoiding wakeup overhead, or skipping updates. The present experiment cannot uniquely attribute old timing differences to those mechanisms.

B. Useful throughput and target time

For measured training time T, we distinguish

$$q_{\text{complete}} = \frac{N}{T}, \qquad q_{\text{accepted}} = \frac{rN}{T}. \tag{13}$$

The output update rate equals q_{complete} in this protocol. An accepted hidden update remains a computational count, not a guarantee of equal optimization value: an old accepted gradient can affect the objective differently from a current one. Nevertheless, reporting it prevents the especially misleading case of treating rejected work as hidden-layer progress.

For target accuracy A, the reported target time is

$$T_A = T_{e^*}, \quad e^* = \min\{e : \text{accuracy}_e \geq A\}. \tag{14}$$

If no recorded checkpoint qualifies, T_A is not reached within the experimental horizon. It is not zero or infinity as a measured value. Checkpoint-based observation leaves the exact crossing time unmeasured. It also does not guarantee that every later checkpoint remains above target. A run that reaches 90% and later dips below it still has a first qualifying checkpoint under this definition.

C. Delay bounds and their scope

Every accepted hidden gradient has $a \leq \tau_{\max}$ by construction. A further bound in this particular FIFO runtime is $a \leq I-1$ for every returned gradient. At the example's forward pass, there are at most $I - 1$ other outstanding examples. The single output owner processes activations in their forward order, and its returning gradients preserve that order in the mixed FIFO inbox. Later activations cannot have their hidden gradients applied before this example. Only earlier outstanding gradients can advance its hidden version before it returns, giving the bound.

This argument depends on FIFO ordering and one output producer. It would not automatically hold with out-of-order output execution, priorities, multiple output workers, retries, or a replay scheme. In-flight admission alone is not a general bound on age for arbitrary asynchronous systems. The new

logs obey the bound, and all accepted ages obey their configured gate.

For $I = 8$, delay limits eight and sixteen both exceed the reachable age of seven. Their acceptance policies are therefore equivalent in this runtime. Separate runs can still differ because thread scheduling changes the sequence of forwards relative to hidden updates. A difference between their measured accuracies cannot be attributed to one bound accepting ages that the other rejects. The pilot data include numeric delay-sixteen runs. They are reported separately because they contain one epoch, rather than the five checkpoints in the main comparison.

D. A conditional optimization intuition

For an illustrative smooth full-model objective with Lipschitz gradient constant L, bounded gradient norm G, constant step η, and a intervening updates,

$$\|\nabla F(\theta_t) - \nabla F(\theta_{t-a})\| \le L\|\theta_t - \theta_{t-a}\| \le L\eta G a. \quad (15)$$

This shows why delay and step size can interact: more parameter movement permits greater derivative mismatch. It is a conditional bound derived from the stated assumptions, not a convergence theorem for our implementation. ReLU has nonsmooth points; the two owners use different versions; learning rate is constant; and age-dependent rejection may change the sampled updates. We do not establish the assumptions needed to transfer a published stochastic convergence guarantee.

Rejection also changes the effective training stream. Since worker service and scheduling can depend on which ReLU units are active, acceptance need not be independent of the input. Even when the coordinator presents a fixed shuffled dataset, the hidden layer may learn from a scheduling-dependent subset. That is another reason not to equate an accepted update with an unbiased stochastic gradient at the current model.

E. Memory and queue costs

The parameter payload is $25{,}450 \times 4 = 101{,}800$ bytes because the implementation stores weights and biases as floats. A cached input has $784 \times 4 = 3{,}136$ bytes of element payload. At $I = 8$, pending input payload is at most 25,088 bytes, before map nodes, vector objects, IDs, and allocator overhead. An activation or returned derivative has 128 bytes of vector payload.

The loader retains the full official training and test image arrays as bytes, totaling $70{,}000 \times 784 = 54{,}880{,}000$ pixel bytes, plus labels. It selects indices into those arrays. Input normalization allocates a float vector for each submitted example, which is also copied into the pending map. Dense backward computation allocates weight, bias, and input-gradient vectors. Such temporary allocations contribute work that a parameter-count calculation alone does not describe.

Retained input state grows as $O(ID)$ rather than $O(IH)$, because the hidden worker stores complete inputs. Model storage grows as $O(DH + HC)$, and the raw dataset storage grows with all loaded records. These are payload and asymptotic calculations, not peak resident-memory measurements. The implementation has no replicated full model or per-version weight snapshot. No claim about cache misses, NUMA behavior, allocator contention, or peak process memory is supported by the present logs.

VII. Experimental method and provenance

A. Controlled workload

The five-epoch studies use 5,000 training examples, 1,000 official test examples, learning rate 0.01, and seed 12345. One persistent `mt19937` first shuffles all training indices and selects 5,000, then shuffles all test indices and selects 1,000. It reshuffles the selected training indices before each epoch. The subset is therefore selected by seeded shuffling, not by taking the first records in file order.

Hidden weights use a normal distribution with scale $\sqrt{2/784}$ and seed 12345. Output weights use scale $\sqrt{1/32}$ and seed 12346. Biases start at zero. Dense arithmetic and stored parameters use floats; loss accumulation and several timing/statistical totals use doubles. The sequential and worker executables share the same dense-layer code and subset-selection procedure.

Standard-library normal sampling and shuffling can differ between toolchains, even when the seed is fixed. A fixed seed also does not fix OS scheduling. The new sequential and synchronous runs match the recorded five-epoch learning values to the printed precision, but this agreement does not identify the original binary, compiler flags, or hardware. We preserve separate timing summaries instead of pooling measurements across builds or dates.

B. Evidence sets

The main recorded epoch CSV contains 45 checkpoint rows across nine five-epoch runs. The comparison CSV contains their final metrics and first-90% checkpoint times. Sequential and delay four each have two observations; the remaining settings have one. Final checkpoints and target times agree between the two CSVs.

The separate worker timing file contains one delay-four and one synchronous five-epoch run. Its human-readable headings are reversed: the block headed synchronous reports $\tau_{\max} = 4, I = 8$, and the block headed delayed reports $\tau_{\max} = 0, I = 1$. We identify the blocks using those numeric fields, retain the raw log, and generate an extracted timing CSV for plotting. These runs are not added to the main CSV averages.

The one-epoch pilot contains 30 delayed runs, five for each limit $0, 1, 2, 4, 8, 16$, and 15 interleaved runs, five each for sequential, synchronous, and full-acceptance delay eight. Its raw text and CSVs are retained. Pilot timing rows are not repetitions of five-epoch rows. Complete build and machine metadata for those early runs were not recorded, so their absolute times remain observations from that stage of the project.

The new sweep uses the submitted source on an Intel Core i5-12450H with 12 logical processors, Windows 11 build 26300, and MSYS2 MinGW GCC 15.2.0. Both C++17 executables are compiled with `-O2 -std=c++17 -pthread`

and warning flags. Metadata records every active source file's SHA-256 hash, both executable hashes, the four decompressed dataset hashes, and each run command. The dataset downloader checks canonical compressed MNIST checksums. No GPU, BLAS library, or neural-network framework is used.

C. Repetitions, scheduling, and metrics

The new sweep comprises three repetitions of eight configurations: sequential, synchronous $(0, 1)$, and delayed $(0, 8), (1, 8), (2, 8), (4, 8), (8, 8), (16, 8)$. Each repetition runs serially in an order shuffled with a separate fixed harness seed. Each executable starts from the same model and data-order seed. Repetitions expose scheduling and timing variation, not independent initialization or dataset variation.

The experiment does not set affinity or control background load. Power state, CPU frequency, thermals, and OS wakeup behavior are not profiled. Runs last only a few seconds, so observed timing ranges matter. Means are descriptive summaries; plot bands and whiskers represent observed minimum and maximum, not confidence intervals or significance tests.

At every drained epoch, the executable reports average pre-update training loss, test loss, accuracy, and cumulative training seconds. Worker summaries report accepted/discarded hidden counts, output updates, average returned and accepted ages, maximum returned age, and active/input-pop durations. The source does not record per-example age traces, parameter checksums, queue occupancy, or peak pending count. The age gate is checked in code, while the logs provide aggregate observations.

The training duration sums the timed epoch windows and excludes loading, initialization, shuffling, evaluation, and final shutdown. Worker input-pop timers surround the complete `pop` call: they include mutex acquisition, predicate waiting, queue removal, and return overhead. They accumulate across the worker lifetime and can include checkpoint-evaluation gaps. Active timers surround processing and queue pushes. The two worker timers overlap in wall time and have different scopes from the training sum, so they cannot be combined into a CPU utilization budget.

VIII. Analysis of recorded measurements

A. Learning curves and final quality

Figure 3 preserves the learning trajectory rather than using only final checkpoints. Means are taken only within repeated configurations; shaded bands show their observed range. Sequential and synchronous curves have identical loss and accuracy values in the supplied CSV. This supports the documented intent of a one-example handshake, and the shared dense-layer routines explain that agreement.

The strict asynchronous zero-age setting ends at 88.9% after five epochs. Delay one reaches 91.1% only at epoch five, and delay two reaches its first 90% checkpoint at epoch four. Delay four reaches the target at epoch two in both CSV runs. Their final accuracies are 92.1% and 91.1%; their test-loss values are 0.263285 and 0.271865. Delay eight accepts all hidden gradients and finishes at 92.2%. These observations do not form a monotonic relationship between delay and convergence speed.

TABLE III
Recorded primary runs. Values are per-setting means; NR means the target was not reached. n is the observed run count.

Setting	n	Time (s)	Acc. (%)	Accept (%)	T_{90} (s)
Sequential	2	28.279	91.40	100.00	11.128
Sync (0,1)	1	28.614	91.40	100.00	11.536
Delay 0	1	9.997	88.90	12.50	NR
Delay 1	1	11.183	91.10	27.17	11.183
Delay 2	1	15.757	90.50	47.97	12.710
Delay 4	2	20.521	91.60	70.28	8.267
Delay 8	1	26.584	92.20	100.00	10.756

Accuracy is also nonmonotonic within runs. The first delay-four run reaches 90.8% at epoch two, drops to 89.9% at epoch three, and finishes above 92%. Training loss continues to decrease, but that does not force the test decision rate to rise at every checkpoint. Test loss measures confidence as well as classification correctness, so its ordering need not match accuracy exactly.

B. Time, acceptance, and the observed compromise

Table III and Fig. 4 summarize the primary supplied sweep. Mean sequential time is 28.279 seconds and mean delay-four time is 20.521 seconds. The ratio of those descriptive means is 1.378. Their mean first-90% times are 11.128 and 8.267 seconds, respectively, a descriptive ratio of 1.346. The two delay-four runs are both quicker to the target than either recorded sequential run.

Zero-age delayed execution is fastest at 9.997 seconds, but accepts only 12.5% of hidden gradients. Over 25,000 completed examples, that implies 3,125 hidden updates. Its completed rate is about 2,501 examples per second, while its accepted hidden rate is about 313 updates per second. Delay four averages 70.28% acceptance across the two primary runs. Its per-run accepted update rates are about 869 and 845 updates per second. High completed throughput at delay zero therefore does not translate into a higher accepted hidden update rate, let alone faster attainment of useful accuracy.

The synchronous run accepts all updates and reaches the same final quality as sequential training, but its 28.614-second time lies within the 26.922 to 29.635-second sequential range. The data do not establish that adding two worker threads improves this strict execution. Delay eight takes 26.584 seconds, close to the sequential observations, despite its good final accuracy. The observed delay-four compromise combines fewer hidden updates with earlier useful quality. It should not be interpreted as equal-work parallel speedup.

C. Separate worker-instrumented observations

The numeric delayed $(4, 8)$ timing block reports 18.727 seconds, 63.52% acceptance, and 91.0% final accuracy. Its first target checkpoint is 7.472 seconds at epoch two. The synchronous $(0, 1)$ block reports 28.951 seconds, full acceptance, and 91.4% final accuracy, with target time 11.630 seconds.

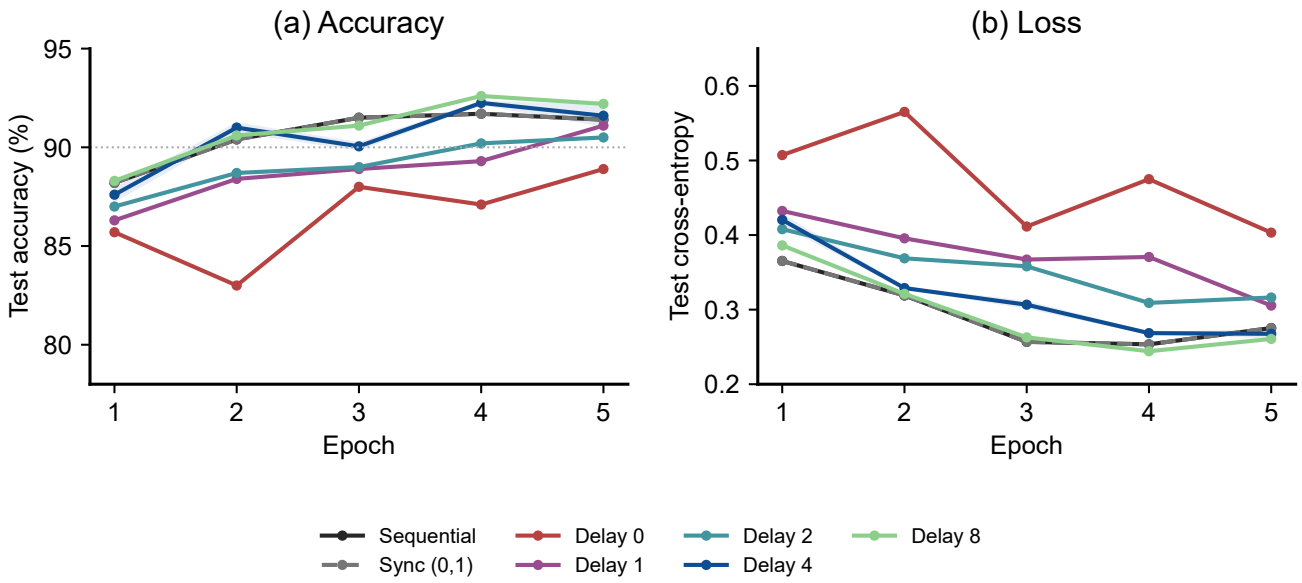


Fig. 3. Recorded test curves. Sequential and synchronous curves coincide in quality. Bands show the range of the two sequential and two delay-four observations; singleton settings have no uncertainty band. The separate instrumented runs are excluded here.

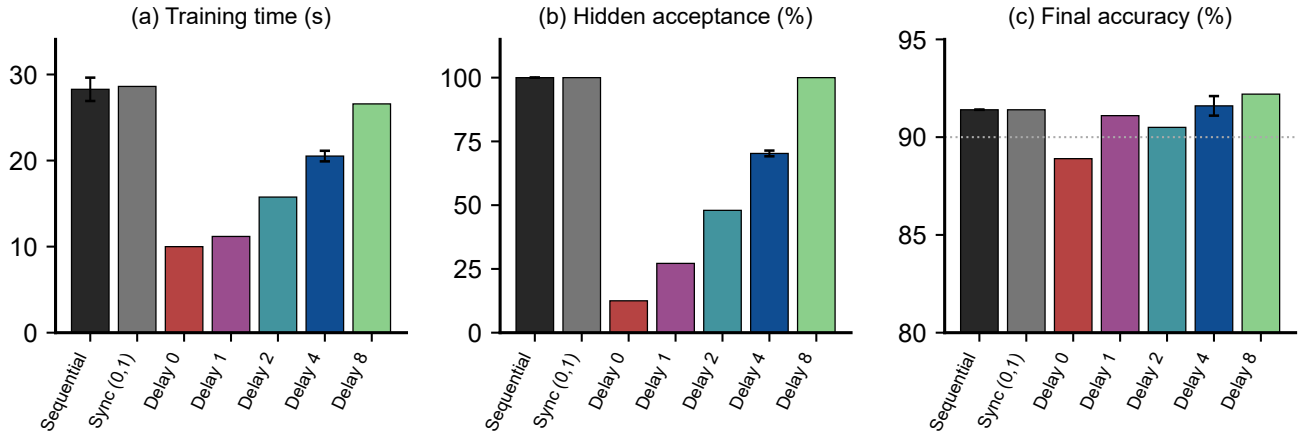


Fig. 4. Recorded elapsed time, hidden acceptance, and final accuracy. Error whiskers show only observed minimum and maximum for repeated settings. Accuracy uses a restricted vertical scale to show differences.

These are consistent with the broad historical pattern, but they remain separate observations.

Worker 0's active path is 18.690 seconds delayed versus 27.146 seconds synchronous; its input-pop duration is 1.102 versus 2.851 seconds. Worker 1's active path is only 0.817 versus 0.586 seconds, while input-pop duration is 18.966 versus 29.421 seconds. The output worker thus spends much more measured time in its input path than its arithmetic path. The small output layer and larger hidden stage give a coherent explanation for the imbalance.

These numbers do not form a partition of wall time. For example, Worker 1's synchronous input-pop duration exceeds the reported 28.951-second training duration. Source inspection confirms that the input timer surrounds the complete queue pop and accumulates between epochs as well as during training. A worker waiting for the coordinator during checkpoint evaluation can accumulate input-pop duration while the epoch training timer is stopped. Exact evaluation and startup contributions are not recorded separately. Summing overlapping per-thread times or turning them into CPU utilization percentages would create a metric the logs never measured. The lower absolute delayed wait is partly associated with a shorter run; it does not establish that the output worker has become well utilized.

IX. New experiments on the submitted implementation

A. *Learning quality and acceptance*

Table IV summarizes the new sweep and Fig. 6 shows every checkpoint. Sequential and synchronous runs have identical

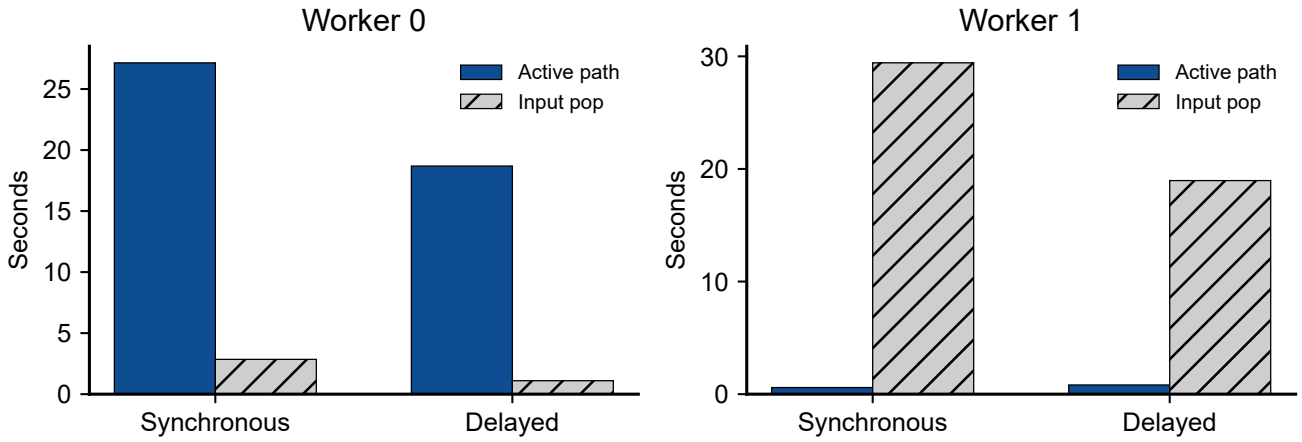


Fig. 5. The two supplied instrumented runs, identified by their numeric configuration. Active-path and input-pop measurements overlap across workers and include different intervals from the training timer; this is not a CPU-utilization chart.

TABLE IV
NEW FIVE-EPOCH SWEEP: THREE SCHEDULING REPETITIONS PER SETTING. T_{90} AVERAGES ONLY REACHING RUNS; DELAY ONE REACHES IN ONE OF THREE RUNS, DELAY ZERO IN NONE, AND EVERY OTHER SETTING IN ALL THREE.

Setting	n	Time (s)	Acc. (%)	Accept (%)	T_{90} (s)
Sequential	3	3.494	91.40	100.00	1.404
Sync (0,1)	3	3.749	91.40	100.00	1.367
Delay 0	3	1.166	86.43	12.88	NR
Delay 1	3	1.555	89.00	29.41	0.822
Delay 2	3	2.058	91.17	48.30	1.513
Delay 4	3	2.408	92.07	75.03	1.199
Delay 8	3	3.476	91.83	100.00	1.328
Delay 16	3	3.790	91.63	100.00	1.456

printed training losses, test losses, and accuracies in all three repetitions. Both finish at 91.4% with test loss 0.275319. Their curves also agree with the recorded five-epoch baseline values. This supports the one-example handshake as a numerical control for the worker protocol.

Hidden acceptance averages 12.88%, 29.41%, 48.30%, and 75.03% for limits zero, one, two, and four. Limits eight and sixteen accept all 25,000 hidden gradients in every run. Zero-age delayed execution finishes between 85.8% and 87.0%; none of those runs reaches 90%. Delay one finishes at 86.7%, 89.4%, and 90.9%, so only its third repetition reaches the target.

Delay two finishes between 90.6% and 91.9%, while delay four finishes at 92.8%, 91.4%, and 92.0%. Their mean final accuracies are 91.17% and 92.07%. Full-acceptance limits eight and sixteen average 91.83% and 91.63%. These values show a cost to very strict rejection, but they do not establish a monotonic accuracy benefit from increasing the delay bound.

The model seed and selected examples are fixed across these runs. Delayed trajectories differ because the hidden owner interleaves forwards and updates according to message arrival. That changes the parameters used for later activations, the age gate's decisions, and the recomputed local mask. An asynchronous setting can therefore vary in quality without any change to its command-line settings.

B. Elapsed time and the baseline

Mean five-epoch times are 3.494 seconds sequentially, 3.749 seconds synchronously, and 2.408 seconds for delay four. The corresponding sequential and delay-four ranges are 2.914 to 4.217 seconds and 2.352 to 2.456 seconds. Every delay-four observation is shorter than every sequential observation in this sweep. The ratio of the descriptive means is 1.451 against sequential and 1.557 against synchronous training.

Delay four omits about 24.97% of hidden updates. Its elapsed gain therefore cannot be assigned entirely to overlapping comparable work. An accepted hidden backward call recomputes a large matrix-vector product and allocates gradient buffers; a rejected call skips that work. The small output layer limits how much arithmetic can be overlapped even when both workers have ready tasks. No equal-work sequential control replays the accepted-example sequence in this experiment.

Synchronous time spans 3.374 to 3.956 seconds and overlaps the sequential range. The means suggest an overhead for the strict queue handshake, but these short, uncontrolled measurements do not isolate its cost precisely. Full-acceptance delay eight averages 3.476 seconds, close to the sequential mean; delay sixteen averages 3.790 seconds. There is no consistent large gain when both layers perform all updates.

The new times are substantially below the recorded 20–29 second values for the longer-work settings. The new source, compiler, and host are known, while the recorded executable and original build flags are not. Compiler optimization, host load, machine behavior, and code revision could contribute. The present data do not identify their individual effects, so timing values from the two dates remain separate.

C. Time to useful quality

The new sequential mean first-90% checkpoint is 1.404 seconds. The corresponding means are 1.367 seconds synchronously, 1.513 seconds at delay two, and 1.199 seconds at delay four. Limits eight and sixteen give 1.328 and 1.456

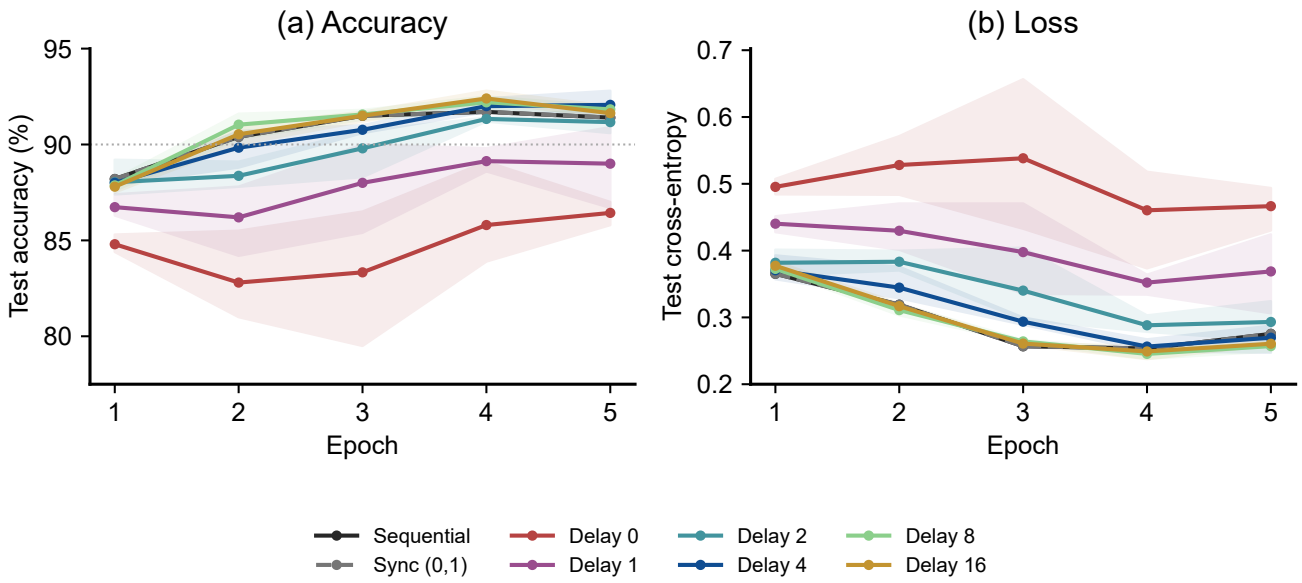


Fig. 6. New test curves from the submitted implementation. Means and bands use three fixed-seed scheduling repetitions. The band is the observed range.

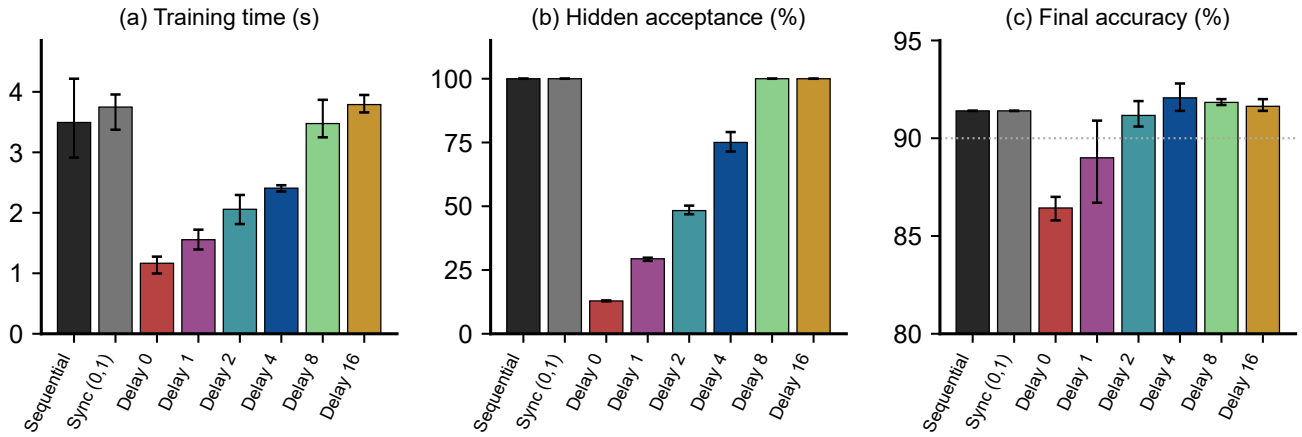


Fig. 7. New elapsed time, hidden acceptance, and final accuracy. Whiskers show the three observed ranges. Faster rejection-heavy runs perform less hidden work; accuracy uses a restricted vertical scale.

seconds. These are checkpoint training times, excluding evaluation.

The fastest completion setting, delay zero, averages 1.166 seconds but never reaches the threshold. Its throughput advantage is not a target-time result. Delay one has a mean target time based on just one reaching observation; comparing that mean with three-run means without its reach count would favor a selectively observed outcome. Delay two and four reach in all three runs. Delay four improves on the sequential mean target time, while delay two takes longer; their checkpoint ordering also differs between repetitions.

The 90% threshold is a compact comparison criterion, not a proof of sustained quality. A later checkpoint can dip below it. Changing the target can also change the ranking: a setting that reaches 90% quickly may finish with poorer loss or fail at a stricter target. The report therefore includes complete curves, final quality, accepted counts, and target reach counts alongside elapsed time.

D. Age and count checks

The new dataset contains 120 epoch rows and 24 final summaries. Every worker summary satisfies accepted plus discarded equals 25,000, and output updates equals 25,000. The synchronous maximum age is zero. Delayed maximum ages are one, two, three, and five for limits zero, one, two, and four, and seven for limits eight and sixteen. Rejected messages are included in the maximum, so an observed maximum may exceed the acceptance limit.

At limit zero, the mean accepted age is zero in all runs. At limits eight and sixteen, full acceptance makes the returned and accepted mean ages equal. Both settings admit every reachable

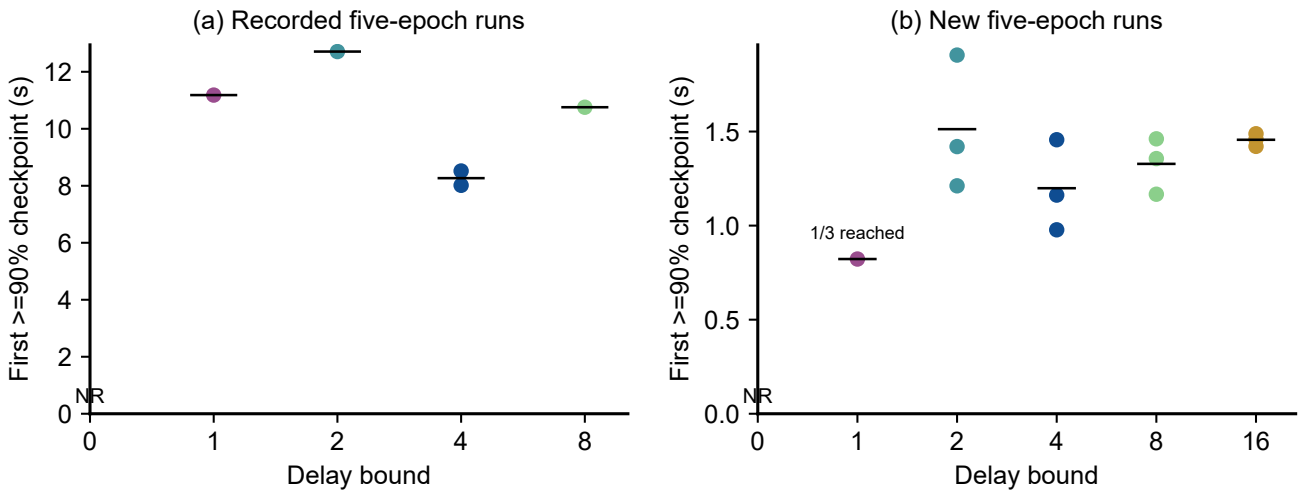


Fig. 8. First recorded 90% checkpoints for delayed settings. Dots are reaching runs and short ticks are their means. NR means no run reaches. New delay one has one reaching run out of three; all its nonreaching runs remain in final-time and accuracy summaries.

age because the FIFO bound is seven at $I = 8$. Differences in their curves consequently reflect scheduling rather than a policy that accepts different ages.

Worker summaries contain elapsed active-path and complete input-pop times, without per-example traces or queue-depth sampling. They are useful for recognizing stage imbalance, but do not measure CPU utilization, context switches, memory bandwidth, or actual time blocked on full queues. The source-level bounds and the aggregate measurements answer different questions and are kept distinct.

X. Discussion: What the Operating Systems Experiment Shows

A. Synchronization granularity matters

The results distinguish the cost of using threads from the benefit of overlapping work. With one example in flight, workers serialize small tasks through queue operations. That provides a useful controlled baseline for inter-thread coordination, but it is a poor assumption for demonstrating CPU speedup. The synchronous implementation reproduces the sequential learning path. Its timing range overlaps the sequential range on the new host, so the sweep does not precisely measure handshake overhead, despite the extra queue operations in its execution path.

Allowing eight examples in flight amortizes some handoff costs across ready work. The hidden worker can process several training messages while gradients are pending. Yet the output stage remains much smaller than the hidden stage, and the hidden owner's serial stream includes both forwards and accepted updates. There is no reason to expect two workers to provide a twofold gain when one stage owns most arithmetic. The design gives overlap opportunities; the partition limits their value.

The project also demonstrates that an age gate affects a system's service demand. As the gate becomes stricter, more results complete without a hidden update. Throughput can rise because the server performs less work, even if no additional arithmetic runs simultaneously. A more demanding evaluation would compare against a sequential implementation that deliberately drops the same number and identity of hidden updates. That control is not present in the current experiments, so we describe the observed modes rather than assigning all improvement to parallelism.

B. Ownership simplifies reasoning, but does not erase costs

Single-writer layer ownership removes the need for a parameter lock around every scalar update. It also gives a precise place to increment the version and make an age decision. Race avoidance follows from access discipline and message ordering, not from the mere presence of a mutex somewhere in the program. The coordinator's epoch drain is as important as worker ownership because evaluation otherwise could read weights while an owner modifies them.

Queue synchronization moves the contention boundary to message transfer. In this small MLP, each message represents one example, so fixed synchronization and allocation costs are visible. The pending map and variant inbox favor clarity over maximal throughput. A production design might preallocate slots, replace the map with a ring indexed by bounded IDs, or combine several examples per message. Such changes could improve performance while altering age distributions and granularity. They would require new measurements rather than a prediction of speedup based only on fewer allocations.

C. Version age captures only one kind of freshness

An accepted-update counter has a useful physical interpretation: it counts how often the hidden parameters changed after an activation. It does not directly quantify the magnitude of those changes. Four small updates and four large updates both produce age four. The same delay bound may therefore behave

differently under another learning rate, optimizer, data order, or model width.

Rejected gradients also do not advance this counter. A run can process many examples while the hidden version moves slowly, so low version age does not imply low wall-clock latency. Conversely, a fast sequence of accepted updates can make a gradient old even if few microseconds pass. Age, queue residence, and model-distance measures could be recorded together in a later study. The present implementation deliberately uses the simpler version policy and reports its limitations.

D. What the one-epoch pilot adds

The pilot sweep provides five observations for each of six delay bounds. Mean completion time increases from 1.777 seconds at delay zero to 3.968 seconds at delay four and 5.422 seconds at delay eight. Hidden acceptance rises from 12.50% through 64.92% to full acceptance. Final one-epoch accuracy rises from 84.20% to 87.32% and 87.80%, respectively. The pilot thus captures the same omission-versus-quality mechanism at a shorter training horizon.

Delay one and two show more scheduling variation than the zero-age setting in that pilot. Delay one's hidden acceptance ranges from 25.20% to 36.90%, while delay two ranges from 37.50% to 56.06%. A fixed seed does not force a fixed accepted-example stream. These are descriptive within-setting differences, not estimates across independent initializations. The five delay-sixteen runs all accept 5,000 hidden updates, with mean time 5.343 seconds and accuracy 87.80%. They do not improve acceptance beyond delay eight.

The separate interleaved pilot comparison contains five runs of each baseline and delay eight. Mean times are 6.047 seconds sequentially, 6.441 seconds synchronously, and 6.020 seconds at delay eight. The mean sequential-to-delay-eight ratio is about 1.004, with overlapping observed ranges. The full-acceptance delayed setting does not show a substantial speed advantage in that comparison. Its one-epoch accuracy is 87.8% in each recorded run versus 88.5% for both baselines.

These pilot values should not be placed beside five-epoch values without their horizon. Five times a one-epoch duration is not a measured five-epoch run, because scheduling, accepted counts, caching, and machine conditions can vary over time. The pilot's 88.5% baseline checkpoint also differs from the later five-epoch baseline's 88.2% first checkpoint. Without a preserved build snapshot for the pilot, the project cannot attribute that difference to a single cause. The raw pilot logs preserve the observation rather than retrospectively assigning it to the submitted binary.

E. How the evidence fits together

The recorded five-epoch comparison and new sweep both show a useful intermediate-delay setting. Their absolute durations differ, and their delay-four acceptance fractions are not identical. The common pattern is that tight rejection shortens work while weakening hidden learning, and full acceptance does not automatically provide a large gain over a single-thread baseline.

Using the actual project source makes the mechanism inspectable. It also exposes the recomputed local mask, copied pending inputs, and complete queue-pop timing scope. Those details explain why an abstract delayed-SGD equation or a worker-wait percentage would be an incomplete description of this experiment. The result is an empirical study of a particular runtime, whose algorithm and measurements can be read together.

There is no basis for selecting delay four as a universal optimum. It is a reasonable setting in these observed five-epoch comparisons, while delay two has competitive new target times and full-acceptance settings preserve more hidden updates. A practical choice depends on the quality threshold, acceptable update omission, and measured behavior on the intended workload.

XI. Limitations and next experiments

A. Measurement and generalization limits

The experiment uses a small selected fraction of MNIST and a small MLP. The test set has only 1,000 examples, so one prediction changes reported accuracy by 0.1 percentage point. A one-percentage-point gap is ten examples on that subset, not an established population-level improvement. Selecting the delay after inspecting the same test curves also limits the interpretation of that test set as an untouched final evaluation. A later tuning study should select the bound on validation data and reserve an independent test set for the final comparison.

Most main recorded settings have one run, the pilot settings have five, and new settings have three scheduling repetitions with one fixed initialization. This does not support significance tests or claims about robustness across random seeds. The new sweeps run on one CPU host without experiment-set affinity and under unmeasured background load. Frequency scaling, heterogeneous CPU cores, thermal effects, OS wakeups, and cache behavior may influence short durations. We record the environment and ranges but do not attribute variation to a profiler measurement we did not collect.

The submitted implementation specifies queue capacity, numeric types, subset selection, and update order. Original machine settings, binaries, and complete build snapshots for the recorded and pilot measurements remain unrecorded. Source availability does not recover that missing provenance. The new sweep has explicit hashes and commands, and its timing conclusions apply to that measured build and host.

B. Algorithmic and runtime limits

The age policy protects only hidden-layer version distance. The output layer still trains on potentially stale activations and always updates. No correction compensates for that mismatch. The hidden worker recomputes its local preactivation, but performs no replay of the downstream forward path, gradient shrinking, or weight prediction. Constant-step nonconvex training with selective rejection is evaluated empirically; we

do not prove convergence or establish the assumptions of the cited methods.

The single output owner, FIFO channels, and configured queue capacity are central to the safety and age arguments. More workers, deeper stages, work priorities, or out-of-order execution would need a revised protocol. The source handles normal drain and exceptional shutdown, and the tests exercise one worker error. Recovery from arbitrary fault sequences, process crashes, cancellation during an epoch, and distributed message loss are outside this experiment. Those would be additional systems requirements rather than consequences of bounded age.

Measured worker timers are not CPU utilization. Calculated payload memory is not peak resident memory. No profiler, energy measurement, hardware cache counter, context-switch trace, or time-weighted queue-depth trace is included. The artifacts support a queue/runtime analysis at the level stated in the paper, not an explanation of every microarchitectural effect.

C. A focused extension plan

The first follow-up should use longer runs and more scheduling and initialization seeds, with an explicit validation/test split and larger fixed subsets. A warmed-up timing protocol, controlled affinity, and recorded power settings would reduce some short-run variability. Repeating the full sweep rather than only the favorable setting would preserve the quality tradeoff and show whether the observed ordering is stable.

The next control should hold hidden update work comparable across modes. A sequential replay of accepted-example IDs would help separate update omission from overlap. A second control could retain full hidden acceptance while varying the in-flight window, since I affects both available overlap and reachable staleness. The present fixed-I sweep cannot isolate all these effects.

Finally, a stage-balance experiment should change hidden width or partition a deeper MLP into stages with comparable service demand. Record isolated forward/backward times, queue residence, full-buffer waits, process CPU time, peak memory, and scheduler traces with clearly stated scopes. Those measurements would test whether delayed gradients improve a balanced pipeline rather than mainly avoiding synchronization overhead around a tiny output stage. Adaptive delay and gradient shrinking are plausible later variants, but their evaluation should begin only after these controls make the baseline understandable.

XII. Repository and reproducibility

The submission repository is https://github.com/kaustubh-dot/delayed-gradient-cpu-mlp. It contains two executable entry points: `src/sequential_baseline.cpp` and `src/main.cpp`. Both use `dense_layer.cpp` and `mnist.cpp`; the worker executable also uses `worker.cpp`, message definitions, and the bounded queue. Build commands are in the README, with an optional CMake configuration and unit/integration tests.

`results/recorded/` contains the main epoch and comparison CSVs and the separate worker timing log. `results/pilot/` contains the two repeated one-epoch studies as raw text and CSVs. `results/reproduction/` contains the 24 new raw logs, 120 combined epoch rows, final summaries, and environment metadata. `results/summary/` contains values derived for tables and worker-time plots. Each plot uses one declared evidence set, rather than a pooled mixture of runs from different stages.

The dataset downloader verifies compressed MNIST checksums and writes the IDX files with the filenames expected by the source. The dataset and executables are excluded from version control. The experiment script runs the configurations serially in a reproducibly randomized order and refuses to replace existing measurements. A new output directory should be used for later repetitions.

The plotting script exports PDF and SVG vector figures and 300-dpi PNG previews. It generates the LaTeX numerical table rows directly from the comparison CSVs. The report directory contains the IEEEtran source, bibliography, generated table inputs, and final PDF. Results checks verify raw-to-summary consistency, source hashes, epoch sequences, target-time derivation, and basic count/age contracts.

A reproduction should build both executables, run the unit and fixture integration suites, download MNIST, and execute the sweep. Another toolchain may change seeded sampling or floating-point details, and another schedule may change the accepted stream. The meaningful comparison is therefore the protocol's invariants and the stated quality/time tradeoff, alongside the actual run values. A discrepancy should be recorded and investigated rather than replaced by an expected number.

XIII. Conclusion

Bounded delayed-gradient execution changes both scheduling and the effective learning stream in this two-stage CPU MLP. Owner threads make versioned updates explicit. Bounded admission limits retained inputs, while condition-variable queues carry dependencies without busy waiting. An accepted hidden update combines a delayed downstream derivative with the local Jacobian recomputed at arrival. Rejection avoids hidden work while retaining the output update.

The recorded five-epoch comparison and the new optimized sweep both show delay four completing sooner than their sequential observations while retaining useful accuracy. The new ratio of mean training times is about 1.45, with roughly one quarter of hidden updates omitted. The one-epoch interleaved pilot shows little timing benefit for full-acceptance delay eight. Together, these observations support a specific speed-quality tradeoff, not a general equal-work CPU speedup or convergence guarantee.

Choosing a delay requires complete learning curves, accepted counts, target reach rates, and a named baseline. Completed-example throughput alone can reward a setting that fails the required quality. The runtime makes that tradeoff

visible and provides a compact operating-systems experiment that can be reproduced and extended.

REFERENCES

[1] F. Niu, B. Recht, C. Ré, and S. J. Wright, "HOGWILD!: A lock-free approach to parallelizing stochastic gradient descent," in *Advances in Neural Information Processing Systems*, vol. 24, 2011. [Online]. Available: https://arxiv.org/abs/1106.5730

[2] Q. Ho, J. Cipar, H. Cui, J. K. Kim, S. Lee, P. B. Gibbons, G. A. Gibson, G. R. Ganger, and E. P. Xing, "More effective distributed ML via a stale synchronous parallel parameter server," in *Advances in Neural Information Processing Systems*, vol. 26, 2013, pp. 1223–1231. [Online]. Available: https://proceedings.neurips.cc/paper/2013/file/b7bb35b9c6ca2aee2df08cf09d7016c2-Paper.pdf

[3] Z. Huo, B. Gu, Q. Yang, and H. Huang, "Decoupled parallel backpropagation with convergence guarantee," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 2098–2106. [Online]. Available: https://proceedings.mlr.press/v80/huo18a.html

[4] Z. Huo, B. Gu, and H. Huang, "Training neural networks using features replay," in *Advances in Neural Information Processing Systems*, vol. 31, 2018. [Online]. Available: https://proceedings.neurips.cc/paper/2018/hash/a36b598abb934e4528412e5a2127b931-Abstract.html

[5] H. Zhuang, Y. Wang, Q. Liu, and Z. Lin, "Fully decoupled neural network learning using delayed gradients," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 10, pp. 6013–6020, 2022. [Online]. Available: https://dr.ntu.edu.sg/server/api/core/bitstreams/d8a50737-064e-4a8a-9800-db783e5be7c7/content

[6] A. Xu, Z. Huo, and H. Huang, "On the acceleration of deep learning model parallelism with staleness," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 2088–2097. [Online]. Available: https://openaccess.thecvf.com/content_CVPR_2020/html/Xu_On_the_Acceleration_of_Deep_Learning_Model_Parallelism_With_Staleness_CVPR_2020_paper.html

[7] J. Peng, Y. Xu, Z. Lin, Z. Weng, Z. Yang, and H. Zhuang, "Decoupled neural network training with re-computation and weight prediction," *PLOS ONE*, vol. 18, no. 2, p. e0276427, 2023. [Online]. Available: https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0276427

[8] A. Maranjyan, A. Tyurin, and P. Richtárik, "Ringmaster ASGD: The first asynchronous SGD with optimal time complexity," in *Proceedings of the 42nd International Conference on Machine Learning*, vol. 267, 2025, pp. 43 120–43 139. [Online]. Available: https://proceedings.mlr.press/v267/maranjyan25b.html

[9] A. Maranjyan, "First provably optimal asynchronous SGD for homogeneous and heterogeneous data," 2026, dissertation, arXiv:2601.02523; not a new peer-reviewed conference result. [Online]. Available: https://arxiv.org/abs/2601.02523

[10] Y. Huang, Y. Cheng, A. Bapna, O. Firat, M. X. Chen, D. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen, "GPipe: Efficient training of giant neural networks using pipeline parallelism," in *Advances in Neural Information Processing Systems*, vol. 32, 2019. [Online]. Available: https://proceedings.neurips.cc/paper/2019/file/093f65e080a295f8076b1c5722a46aa2-Paper.pdf

[11] M. Jaderberg, W. M. Czarnecki, S. Osindero, O. Vinyals, A. Graves, D. Silver, and K. Kavukcuoglu, "Decoupled neural interfaces using synthetic gradients," in *Proceedings of the 34th International Conference on Machine Learning*, vol. 70, 2017, pp. 1627–1635. [Online]. Available: https://proceedings.mlr.press/v70/jaderberg17a.html

[12] S. U. Stich, "Local SGD converges fast and communicates little," in *International Conference on Learning Representations*, 2019. [Online]. Available: https://arxiv.org/abs/1805.09767

[13] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. [Online]. Available: https://leon.bottou.org/papers/lecun-98h